# Who Wrote It? Identifying LLMs from Their Responses

**CMPSC 448 - Midterm Project**
Nihat Hasan | Individual participant and team leader
October 11, 2026

## 1. Problem and research questions

This project tests whether a small neural text classifier can identify which language model generated a response. It compares a convolutional neural network (CNN) and a recurrent neural network (LSTM) using public outputs from three model families. The task is three-class attribution among the selected models, not human-versus-AI detection.

**RQ1:** Can the model that generated a response be identified using the response alone?
**RQ2:** Does adding the user prompt improve attribution compared with response-only classification? Prompt-only classification provides a control condition.

The strongest result was 70.25% test accuracy for the response-only CNN, compared with the balanced chance level of 33.33%. The response-only LSTM achieved 49.86%. Adding the prompt did not consistently improve performance across architectures.

## 2. Dataset construction

The data were downloaded from the public AlpacaEval all-outputs file [1, 2]. The source fields instruction, output, and generator provide the required prompt, response, and generating-model label. The selected labels were gpt4, claude, and vicuna-13b, representing the GPT, Claude, and LLaMA-derived Vicuna families. These are historical dataset labels, not claims about current commercial model versions. Exact underlying model revisions and original generation settings were not independently recovered.

Rows with missing or empty prompts or outputs were removed. Duplicate (prompt, generator) pairs were removed, retaining the first row. Only prompts with a response from all three selected models were retained. This produced **805 shared prompts and 2,415 responses**, with 805 examples per class. No new LLM responses were generated for this project.

| Split | Unique prompts | Responses | Responses per class |
|---|---|---|---|
| Training | 563 | 1689 | 563 |
| Validation | 121 | 363 | 121 |
| Test | 121 | 363 | 121 |

Prompts were split approximately 70/15/15 with random seed 42. Every response to a particular prompt stayed in the same split. This avoids exact-prompt leakage across training and test sets. Semantically similar prompts and cross-prompt response duplicates were not separately audited. The repository links to the source instead of republishing the raw dataset.

# 3. Preprocessing and model implementation

Text was lowercased and tokenized with a regular expression that keeps word sequences and individual punctuation characters. Whitespace, including newline characters, was discarded. A shared vocabulary was built from training prompts and responses only: the 12,000 most frequent tokens plus PAD, UNK, and SEP, for 12,003 entries. Unknown tokens map to UNK; PAD has index zero. Embeddings were learned from scratch.

| Input setting | Sequence supplied to classifier |
|---|---|
| Prompt only | First 96 prompt tokens; padded to 96. |
| Response only | First 160 response tokens; padded to 160. |
| Prompt + response | Up to 96 prompt tokens, SEP, then up to 160 response tokens; padded to 257. |

The response token budget is the same in the response-only and combined settings. Thus, including a prompt does not reduce the retained response length. The shared vocabulary also stays fixed across settings. Labels, model names from the generator field, and split identifiers are never appended to input text.

**CNN.** A 64-dimensional embedding feeds three parallel one-dimensional convolutions with kernel widths 3, 4, and 5. Each convolution has 64 filters, followed by ReLU and max pooling over time. Windows that begin entirely in padding are masked before pooling; boundary windows may include some padding. The pooled features are concatenated into 192 values, followed by dropout (0.3) and a linear layer producing three logits.

**LSTM.** A 64-dimensional embedding feeds a single-layer, unidirectional LSTM with hidden size 64. The hidden state at the final real token is selected, rather than the padded sequence ending. Dropout (0.3) and a linear layer produce three logits. This is the required recurrent model.

# 4. Training and evaluation protocol

| Parameter | Setting |
|---|---|
| Optimizer / loss | Adam, learning rate 0.001 / cross-entropy |
| Training budget | 8 epochs for each of 6 independent models; batch size 64 |
| Gradient clipping | Maximum gradient norm 1.0 |
| Randomness / runtime | Seed 42; CUDA in Google Colab |
| Checkpoint selection | Highest validation macro-F1; earliest epoch retained on an exact tie |
| Test metrics | Accuracy and macro-F1, on the same 363 test responses |

All six configurations were specified before the results were inspected. The test set was evaluated after checkpoint selection for each configuration. Accuracy measures the fraction of correct labels; macro-F1 averages per-class F1 scores equally. The recorded script runtime was about 22.8 seconds, including download and output generation. GPU model and dependency versions were not recorded; exact bitwise reproduction across environments is not guaranteed.

# 5. Results: RQ1 and RQ2

| Model | Input setting | Test accuracy | Macro-F1 | Best epoch |
|---|---|---|---|---|
| CNN | Prompt only | 33.33% | 0.3130 | 2 |
| LSTM | Prompt only | 33.33% | 0.3231 | 3 |
| CNN | Response only | 70.25% | 0.7044 | 8 |
| LSTM | Response only | 49.86% | 0.4948 | 8 |
| CNN | Prompt + response | 65.84% | 0.6633 | 5 |
| LSTM | Prompt + response | 51.24% | 0.5090 | 8 |

**RQ1:** The response-only CNN correctly classified 255 of 363 responses (70.25%), compared with 181 of 363 (49.86%) for the LSTM. Both exceeded 33.33% chance accuracy. The CNN advantage was 20.39 percentage points in this run. A plausible explanation is that local wording and punctuation patterns are useful for attribution, whereas the final-state LSTM has to compress the retained sequence into one vector. This is an interpretation, not a demonstrated causal explanation.

**Prompt-only control:** Both models achieved exactly 33.33% accuracy. Each held-out prompt appears three times with identical input text and three different labels. A deterministic classifier makes the same prediction for all three copies, so exactly one prediction is correct. Prompt-only accuracy is therefore fixed at one-third by this balanced construction. Macro-F1 can still differ because the models predict different class distributions.

**RQ2:** Adding prompts reduced CNN accuracy from 70.25% to 65.84% (-4.41 percentage points) and increased LSTM accuracy from 49.86% to 51.24% (+1.38 points). These results provide no consistent benefit from prompt inclusion. CNN validation macro-F1 favored the combined setting, but the test ranking reversed, showing why validation improvement alone does not establish generalization improvement.

The combined CNN selected epoch 5. The displayed validation values at epochs 5 and 8 both round to 0.711, but selection used unrounded values. Other selected epochs are listed in the table.

**Paired uncertainty analysis.** A post-hoc bootstrap resampled the 121 test prompts with replacement 10,000 times, keeping each prompt's three labels together. The approximate 95% percentile intervals for the combined-minus-response-only accuracy difference were -8.54 to 0.00 percentage points for CNN and -0.55 to +3.31 points for LSTM. These intervals reflect test-prompt sampling only, not training-seed variability. They do not establish a reliable improvement from adding prompts.

# 6. Error analysis and descriptive patterns

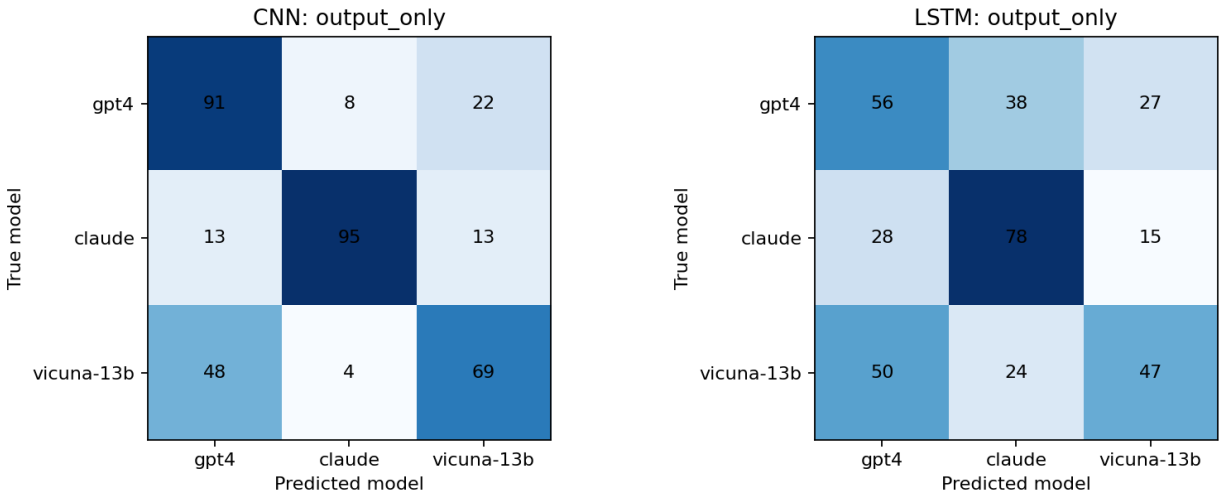



Figure 1. Response-only test confusion matrices. Rows are true labels and columns are predicted labels; each true class has 121 examples.

For the CNN, recall was 78.51% for Claude (95/121), 75.21% for GPT-4 (91/121), and 57.02% for Vicuna (69/121). Its largest error cell was Vicuna predicted as GPT-4: 48 cases. This identifies the hardest distinction in this run, but does not establish why the two output distributions overlap. For the LSTM, recall was 64.46% for Claude, 46.28% for GPT-4, and 38.84% for Vicuna.

**Exploratory RQ4: observable text characteristics.** The following means were computed from training responses only, using their full text rather than the truncated classifier input. Words are whitespace-separated; type-token ratio is unique regex tokens divided by all regex tokens.

| Model | Words | Characters | Newlines | Type-token ratio |
|---|---|---|---|---|
| Claude | 210.31 | 1,287.82 | 16.43 | 0.5177 |
| GPT-4 | 213.52 | 1,362.42 | 15.34 | 0.5348 |
| Vicuna-13B | 176.03 | 1,047.81 | 9.73 | 0.5020 |

Vicuna responses were shorter on average and had fewer newlines. GPT-4 had the highest average type-token ratio, although this statistic depends on response length and cannot by itself establish richer vocabulary. These differences describe this dataset; they do not prove that a particular feature caused a prediction.

In particular, the tokenizer removes newlines, so the classifier cannot directly use newline counts. Punctuation and retained lexical patterns remain available. No feature-removal ablation or saliency study was conducted. A useful follow-up would compare punctuation removal, length-matched samples, and normalized formatting, using validation data for experiment choices.

# 7. Limitations

**Coverage.** Only three historical model labels and one benchmark were studied. The results should not be extrapolated to new model revisions, unseen model families, or arbitrary real-world text. No cross-domain RQ3 experiment was performed.

**Training budget.** There was one seed, eight epochs, and no broad hyperparameter search. The models used random embeddings, and the LSTM used only its final state. Its weaker result does not show that recurrent models are inherently unsuitable for attribution.

**Input representation.** Truncation discards content beyond 96 prompt and 160 response tokens, and whitespace removal discards formatting. Explicit model self-identification inside responses was not systematically audited or masked. Such text, benchmark artifacts, and differences in original prompting or generation settings may contribute to apparent fingerprints.

**Data and reproducibility.** Grouping prevents exact-prompt overlap but does not eliminate every possible semantic overlap. Source data were downloaded from a mutable main-branch URL. A checksum of the processed dataset, original code, metrics, and prediction labels are preserved, but source changes and unpinned packages may affect future reruns. Public availability alone should not be treated as unrestricted redistribution permission; raw text is not bundled in the repository.

# 8. Lessons and experience

The prompt-only result illustrates how dataset construction changes the meaning of an experiment. With a shared, balanced prompt set, the prompt cannot distinguish the three labels. If prompt-only classification performed well on a differently collected dataset, unequal prompt topics or collection procedures would need to be checked before interpreting it as an LLM fingerprint.

The experiment also shows why adding information does not guarantee better held-out performance. Combining prompts and responses adds context, but also changes the sequence and the learning problem. The CNN did better with responses alone in this run. Validation-based checkpoint selection, prompt-grouped splitting, and a clearly separated test set made that comparison more informative.

Saving predictions, training histories, and configuration details allowed the reported numbers to be checked after training. A stronger next version would repeat multiple seeds, audit model-name disclosures, add causal feature-removal tests, and evaluate new domains and model versions.

# 9. Reproducibility, assistance, and references

The accompanying repository contains the runnable training code, the original Colab script, all six result rows, training histories, confusion matrices, hashed prompt identifiers with test labels and predictions, and the paired bootstrap script. ChatGPT assisted with code drafting, debugging guidance, analysis, and report drafting. The experiments were executed in Google Colab. The author is responsible for reviewing and explaining the submitted work.

[1] Tatsu Lab. AlpacaEval dataset and precomputed model outputs. Accessed October 11, 2026. https://huggingface.co/datasets/tatsu-lab/alpaca_eval

[2] Tatsu Lab. AlpacaEval project repository. https://github.com/tatsu-lab/alpaca_eval